

Proof Buddy — Setup Guide

This document explains how to set up and run the Proof Buddy project locally for development, and how to deploy it using Docker.

1. Prerequisites

You need the following installed before starting:

Dependency	Version	Notes
Node.js	18+ (LTS recommended)	Required to run the React frontend
npm	Bundled with Node.js	Package manager for React
Python	3.12.x	Required for Django backend
pip	Bundled with Python	Package manager for Python
MySQL	8.x	Database server
Git	Any recent version	Source control
Docker + Docker Compose	Any recent version	Only needed for Docker/production workflow

2. Local Development Setup (Without Docker)

This is the recommended approach for active development and debugging.

Step 1: Clone the repository

```
git clone <repository-url>
cd proof-buddy-New
```

Step 2: Configure the environment file

The backend requires a `.env` file at the project root (`proof-buddy-New/.env`). Create it with the following variables:

```
# Django settings
SECRET_KEY=your-django-secret-key-here
DEBUG=True

# Database connection (MySQL)
DB_NAME=proofbuddy
DB_USER=your_mysql_username
DB_PASSWORD=your_mysql_password
```

```
DB_HOST=localhost
DB_PORT=3306

# Email (SMTP) – for password reset functionality
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_HOST_USER=your-email@gmail.com
EMAIL_HOST_PASSWORD=your-gmail-app-password
EMAIL_USE_TLS=True

# Frontend URL (used in password reset emails)
FRONTEND_URL=http://localhost:3000
```

Generating a Django SECRET_KEY: Run `python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"` to generate one.

Gmail App Password: If using Gmail for email, you must generate an "App Password" in your Google Account security settings (not your regular Gmail password).

Step 3: Set up the MySQL database

In your MySQL client (e.g., MySQL Workbench, `mysql` CLI), run:

```
CREATE DATABASE proofbuddy CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'proofbuddy_user'@'localhost' IDENTIFIED BY 'your_password';
GRANT ALL PRIVILEGES ON proofbuddy.* TO 'proofbuddy_user'@'localhost';
FLUSH PRIVILEGES;
```

Alternatively, see [database/setup.sql](#) for a pre-written setup script.

Step 4: Set up the Python backend

```
cd django_server

# Create and activate a virtual environment
python -m venv venv
venv\Scripts\activate          # Windows
# source venv/bin/activate     # macOS/Linux

# Install dependencies
pip install -r requirements.txt

# Run database migrations (creates all tables)
python manage.py migrate

# Create the cache table (required by Django DB cache)
python manage.py createcachetable
```

```
# (Optional) Create an admin superuser
python manage.py createsuperuser
```

There is a convenience VS Code task for setup: **djangoSetup** (runs `migrate` + `createcachetable`).

Step 5: Set up the React frontend

```
cd client

# Install JavaScript dependencies
npm install
```

Step 6: Run both servers

You need two terminals running simultaneously.

Terminal 1 — Django backend:

```
cd django_server
python manage.py runserver
```

The backend starts at `http://localhost:8000`.

Terminal 2 — React frontend:

```
cd client
npm start
```

The frontend starts at `http://localhost:3000` and automatically proxies `/api/` requests to the Django backend.

There are also VS Code tasks configured for this: **Django Server** and **React Dev Server**, or **Start All Servers** to launch both at once.

On Windows, the convenience script `start-dev.bat` (or `start-dev.ps1`) can start both servers.

Step 7: Access the application

Open `http://localhost:3000` in a browser. Create an account (you can bypass email verification in development if `is_active = True` is the default in `AccountManager.create_user`, which it currently is for development).

3. Running Tests

Backend Django tests

```
cd django_server
python manage.py test equational_reasoning_api
python manage.py test induction_api
python manage.py test accounts
python manage.py test proofs
```

Or run all tests at once:

```
python manage.py test
```

Expression engine unit tests

```
cd django_server
python -m pytest expression_tree/testApplyRule.py
python expression_tree/runTests.py
```

Integration tests (standalone)

```
# From the project root, with the Django server running:
python test_equational_api.py
```

Frontend tests

```
cd client
npm test
```

4. Docker Deployment

The Docker Compose setup runs three containers: the Django backend, the React frontend, and Nginx as a reverse proxy.

Step 1: Prepare environment variables

The `.env` file must exist at the project root (same location as `docker-compose.yml`). Use production values:

```
SECRET_KEY=strong-random-production-key
DEBUG=False
```

```
DB_NAME=proofbuddy
DB_USER=db_user
DB_PASSWORD=strong_db_password
DB_HOST=your_mysql_host          # External MySQL host or a separate DB container
DB_PORT=3306
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_HOST_USER=your-email@example.com
EMAIL_HOST_PASSWORD=app_password
EMAIL_USE_TLS=True
FRONTEND_URL=https://yourdomain.com
```

Step 2: Set the backend URL for the React build

In `docker-compose.yml`, update the `REACT_APP_BACKEND_API_BASE_URL` build argument to point to your API:

```
client:
  build:
    context: ./client
    args:
      REACT_APP_BACKEND_API_BASE_URL: "https://api.yourdomain.com"
```

Step 3: Build and start all containers

```
docker-compose up --build
```

This will:

1. Build the Django container (installs pip packages, runs `entrypoint.sh` which runs migrations and starts Gunicorn).
2. Build the React container (runs `npm run build`, serves via Nginx).
3. Start Nginx (serves static Django files and proxies API calls to Django).

Step 4: Verify services

Service	URL
React frontend	http://localhost:9090
Django/Nginx	http://localhost:9091
Django admin	http://localhost:9091/admin

Stopping and removing containers

```
docker-compose down
```

To remove volumes as well (this will destroy cached static files and logs):

```
docker-compose down -v
```

5. Environment Variables Reference

Variable	Required	Description
<code>SECRET_KEY</code>	Yes	Django cryptographic key. Must be long, random, and secret.
<code>DEBUG</code>	Yes	<code>True</code> for development, <code>False</code> for production.
<code>DB_NAME</code>	Yes	MySQL database name.
<code>DB_USER</code>	Yes	MySQL username.
<code>DB_PASSWORD</code>	Yes	MySQL password.
<code>DB_HOST</code>	Yes	MySQL host (e.g., <code>localhost</code> or Docker service name).
<code>DB_PORT</code>	Yes	MySQL port (default: <code>3306</code>).
<code>EMAIL_HOST</code>	Yes	SMTP server hostname.
<code>EMAIL_PORT</code>	Yes	SMTP port (587 for TLS).
<code>EMAIL_HOST_USER</code>	Yes	SMTP login username (email).
<code>EMAIL_HOST_PASSWORD</code>	Yes	SMTP password or app password.
<code>EMAIL_USE_TLS</code>	Yes	<code>True</code> to use STARTTLS.
<code>FRONTEND_URL</code>	Yes	Full URL to the frontend (used in email links).
<code>REACT_APP_BACKEND_API_BASE_URL</code>	Docker only	Backend API base URL; baked into React build at build time.

6. Development Workflow Tips

When modifying backend proof logic

After changing Python files in `expression_tree/`, no server restart is needed if the Django dev server is running with `--reload` (the default). Changes are picked up automatically.

When changing Django models

Every model change requires a new migration:

```
python manage.py makemigrations
python manage.py migrate
```

When modifying the frontend

The React dev server hot-reloads automatically. The only exception is changes to `.env` variables, which require a server restart.

Resetting the database during development

If you need a clean slate:

```
python manage.py flush          # Deletes all data, keeps tables
# OR
python manage.py reset_db      # Drops and recreates tables (requires django-
extensions)
```

Django Admin

Navigate to <http://localhost:8000/admin> (with the dev server running) to inspect or modify database records directly. You need a superuser account (`python manage.py createsuperuser`).

Cache management

The Django cache uses a database table called `django_cache`. To clear all cached proof objects during debugging:

```
python manage.py shell
>>> from django.core.cache import cache
>>> cache.clear()
```